

中國民航大学

《软件工程课程设计》

软件设计规格说明书

基于 Agent 的民航旅客
智能出行系统

项目名称

组长学号姓名: 24XXXXX XXX

成员学号姓名: 24XXXXX XXX

二〇二六年九月

目录

第 1 章 引言.....	1
1.1 软件设计目标和原则.....	1
1.2 软件设计的约束和限制.....	1
第 2 章 软件体系结构设计.....	2
第 3 章 用户界面设计.....	4
3.1 系统界面的外观设计及其类表示.....	4
3.1.1 XX 界面	4
3.1.2 XX 界面	4
3.2 系统界面流设计.....	4
第 4 章 详细设计.....	6
4.1 用例设计.....	6
4.1.1 获取个人行程信息数据用例实现的设计方案.....	6
4.1.2 获取行程安排建议用例实现的设计方案.....	8
4.1.3 获取行程延误风险用例实现的设计方案.....	9
4.1.4 导航到机场用例实现的设计方案.....	10
4.2 类设计.....	11
4.3 数据模型设计.....	13
4.3.1 数据设计类及主要字段.....	13
4.3.2 数据操作设计.....	14
4.4 部署设计.....	14

图目录

图 2-1 基于 Agent 的民航旅客智能出行系统系统体系结构逻辑视图	3
图 3-1 XX 系统 XX 界面外观设计	4
图 3-2 XX 系统 XX 界面类表示	4
图 3-X XX 系统界面流的顺序图	5
图 4-1 系统“获取个人行程信息数据”用例设计顺序图	7
图 4-2 系统“获取行程安排建议”用例设计顺序图	8
图 4-3 系统“获取行程延误风险”用例设计顺序图	9
图 4-4 系统“导航到机场”用例设计顺序图	10
图 4-5 系统设计类图	

表目录

表 4-1 核心设计类及职责	13
表 4-2 核心数据对象及字段	13
表 4-3 软件与外界环境的交互关系	15

第 1 章 引言

1.1 软件设计目标和原则

软件设计的目标是,根据软件系统的需求(包括功能性需求和非功能性需求),综合考虑软件开发过程中的各种制约因素(如技术、资源、进度等),遵循软件工程的设计原则(如模块化、信息隐藏、问题分解等),给出软件系统的实现解决方案和蓝图,产生可指导编码实现的设计模型及文档。

软件设计活动还须遵循相关的策略和原则,以指导软件设计人员的行为,并对设计成果提出约束和要求。具体的,这些设计策略和原则描述如下。

- 1) 抽象和逐步求精的原则
- 2) 模块化与高内聚度、低耦合度的原则
- 3) 信息隐藏的原则
- 4) 多视点以及关注点分离的原则
- 5) 软件重用的原则
- 6) 迭代设计的原则
- 7) 可追踪性的原则

1.2 软件设计的约束和限制

- 运行环境要求: Go 1.25, npm
- 开发语言: Golang, TypeScript, Vue3
- 标准规范: Effective Go
- 开发工具: Visual Studio Code

第 2 章 软件体系结构设计

图 2-1 描述了“基于 Agent 的民航旅客智能出行系统”的设计架构。系统主要由用户界面层、业务逻辑层、数据层、设备接口层和外部系统组成。

用户界面层负责向旅客展示系统信息，并接收旅客的操作和状态上报，承担了边界类“行程创建界面类”“行程状态界面类”和“定位与阶段确认界面类”的职责。其中，行程创建界面类负责录入航班与行李信息并提交行程；行程状态界面类负责展示航班状态、行程时间轴和行动建议；定位与阶段确认界面类负责上报旅客当前位置以及确认当前所处阶段。用户界面层只负责信息展示和用户输入，不负责风险计算和业务判断。

业务逻辑层负责系统核心业务逻辑的处理，承担了“行程编排控制类”“数据感知控制类”“行动决策控制类”和“快照与推送控制类”的职责。行程编排控制类负责创建行程、更新旅客位置或所处阶段，并触发重新分析；数据感知控制类负责获取航班与机场数据、校验观测数据以及评估数据质量；行动决策控制类负责分析旅客阶段与行程风险，并生成行动建议；快照与推送控制类负责保存完整行程快照，并通过实时推送将分析结果发送给用户界面层。业务逻辑层是系统的统一编排中心，按照先获取数据、再进行决策的顺序执行分析流程。

设备接口层负责封装系统与外部数据源及服务之间的交互。该层中的“聚合数据源接口”用于支持系统与“航班与机场数据来源”之间的交互，实现航班状态和机场概况等数据的获取；“大模型接口”用于支持系统与“大模型服务”之间的交互，实现行动建议的生成；“地图导航接口”用于支持系统与“地图导航应用或网页”之间的交互，实现导航应用或导航网页的跳转。系统通过接口层访问外部系统，业务逻辑层不直接依赖具体的外部服务实现。

数据层负责系统运行数据和分析快照的存储，包括“行程实体类”“航段实体类”“行程快照实体类”“行程状态实体类”和“位置进度实体类”。其中，行程实体类用于保存行程标识和行李信息；航段实体类用于保存航班号、出行日期、出发机场和到达机场；行程快照实体类用于保存分析状态、生成时间和数据质量；行程状态实体类用于保存航班状态、时间轴节点、预计路程时长和数据质量；位置进度实体类用于保存旅客经纬度以及手动确认的阶段信息。

图2-1 系统体系结构图

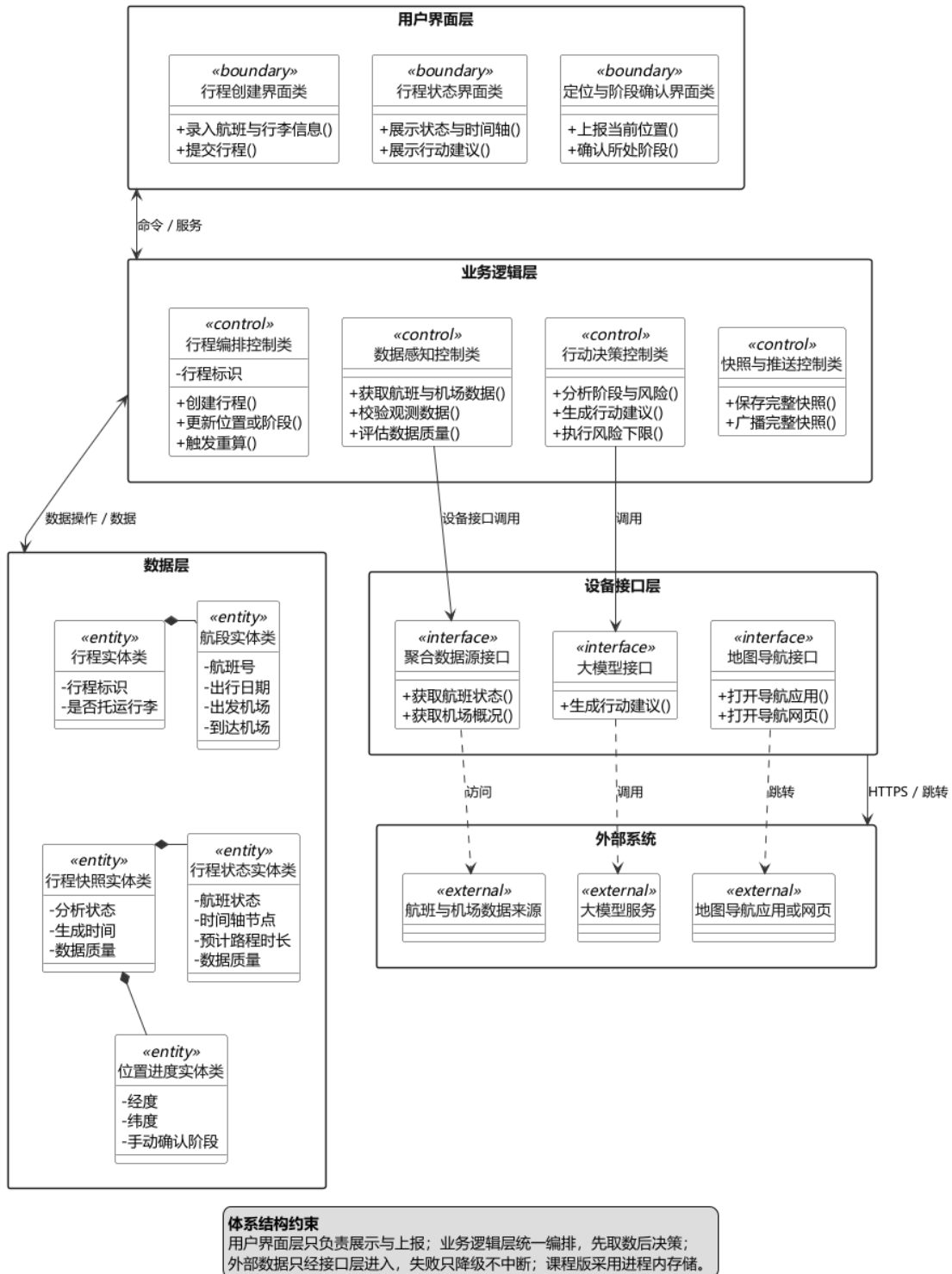


图 2-1 基于 Agent 的民航旅客智能出行系统系统体系结构逻辑视图

第 3 章 用户界面设计

3.1 系统界面的外观设计及其类表示

〈类图中请包含注释部分，阐述清晰动态元素、输入元素、命令元素等〉

〈对于每一个界面，分别用简短语言描述该界面的功能，并在界面上标注各类元素〉

3.1.1 XX 界面

本界面为本软件的 XX 界面。该界面的功能是*****。界面的外观如图 3-1 所示，界面类表示如图 3-2 所示。

图 3-1 XX 系统 XX 界面外观设计

图 3-2 XX 系统 XX 界面类表示

3.1.2 XX 界面

3.2 系统界面流设计

〈用简短语言描述该系统界面流的界面职责及跳转关系〉

根据“空巢老人智能看护系统”的用例描述以及每个用例的交互图，可以发现该软件系统在手机端的 APP 需要有以下一组界面以支持用户的操作。

- 引导界面“GuidingUI”，APP 加载启动时用于展示和介绍该软件系统。
- 登录界面“LoginUI”，其职责是帮助用户输入用户信息以登录到系统之中。

- 监视老人状况界面“**MonitoringUI**”，其职责是显示老人在家的视频、图像和语音等信息。
- 控制机器人运动界面“**MotionCtrlUI**”，其职责是帮助用户操纵机器人的运动。
- 与老人交互界面“**BiCallUI**”，其职责是帮助用户来与老人进行视频和语音交互。
- 系统设置界面“**SettingUI**”，其职责是帮助管理人员配置系统。

系统界面流跳转关系如图 3-XXX 所示。

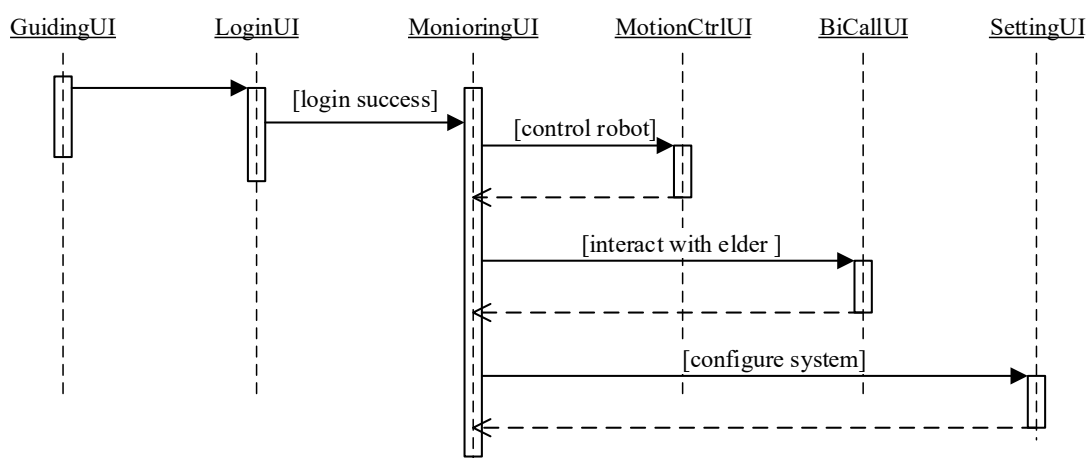


图 3-X XX 系统界面流的顺序图

第 4 章 详细设计

4.1 用例设计

第 2 章确定了旅客直接使用的四个核心用例：获取个人行程信息数据、获取行程安排建议、获取行程延误风险和导航到机场。本节将需求用例转化为可实现的对象协作方案。四幅用例设计顺序图直接复用需求规格说明书中的图 2-2 至图 2-5，保留了用户界面、Gin 路由、数据感知 Agent、行动决策 Agent、行程状态实体以及外部导航服务之间的消息顺序。

4.1.1 获取个人行程信息数据用例实现的设计方案

旅客进入行程页面或主动刷新时，用户界面向 Gin 路由发送获取行程状态的请求。Gin 路由首先读取行程快照；当快照不存在、已经过期或数据质量为降级状态时，路由调用数据感知 Agent 构建最新的 WorldState。数据感知 Agent 检查缺失、过期和冲突字段，调用航班、机场和路线数据技能获取外部数据，并记录数据来源、更新时间、Evidence 和 DataQuality。外部数据查询失败时，系统使用最近一次有效数据，并将相应字段标记为 stale 或 unknown。处理结果保存为新的行程快照后返回用户界面，客户端展示航班、机场、路线、旅客位置以及数据新鲜度。

图2-2 系统“获取个人行程信息数据”用例顺序图

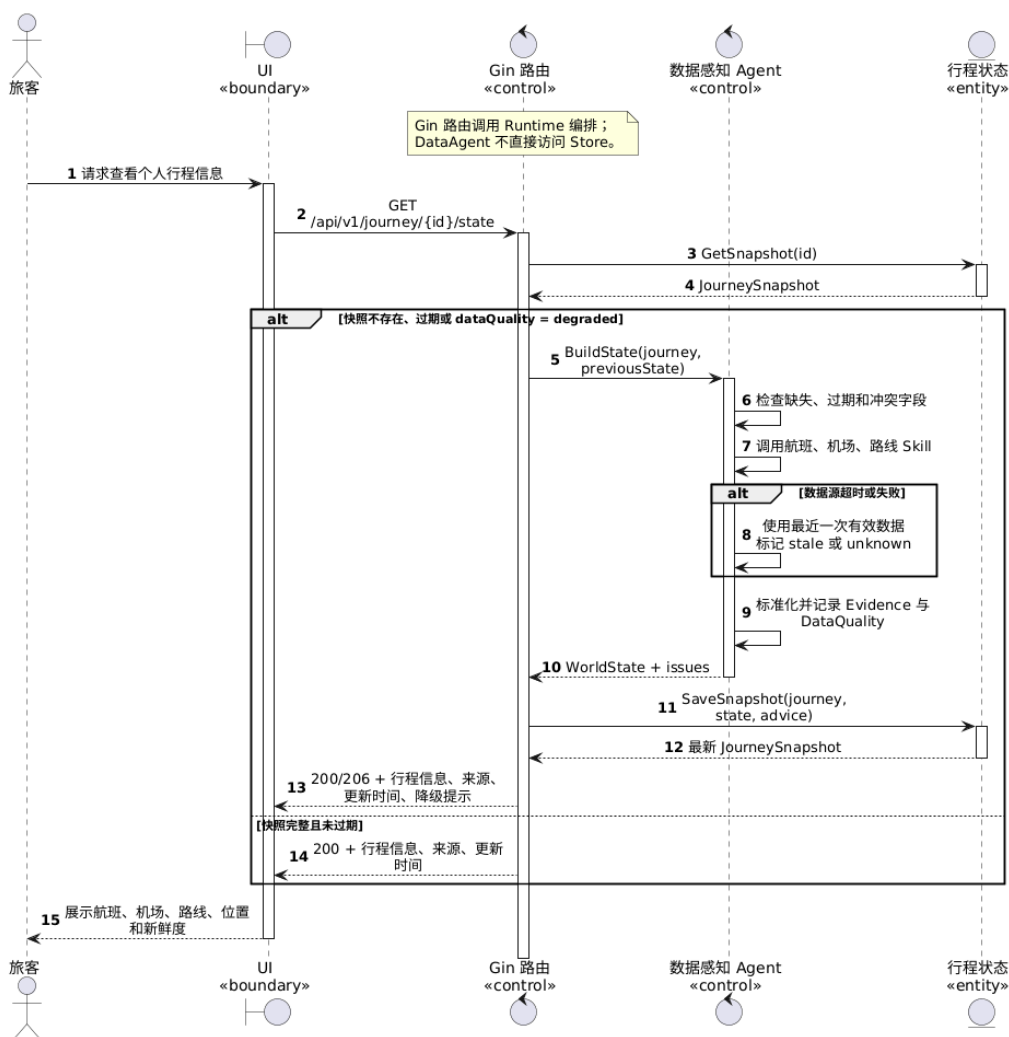
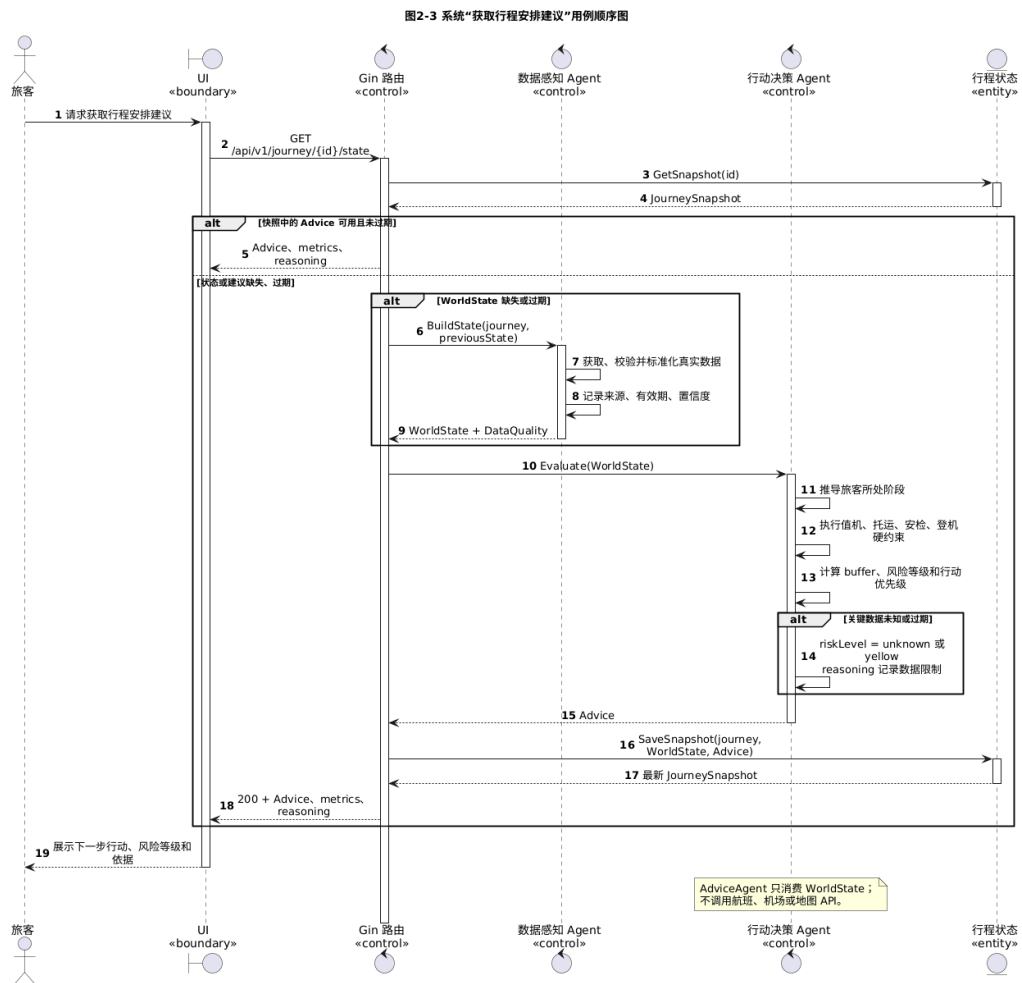


图 4-1 系统“获取个人行程信息数据”用例设计顺序图

如图 4-1 所示，数据感知 Agent 不直接访问快照存储，而是由 Gin 路由统一编排。快照可用且未过期时，系统直接返回已有结果；快照不可用或已降级时，系统重新获取并标准化数据。即使某个数据源超时或失败，系统仍返回其他可用信息，并向客户端说明数据限制。

4.1.2 获取行程安排建议用例实现的设计方案

旅客查看行动建议，或航班状态、机场状态、旅客位置等信息发生变化后，用户界面向 Gin 路由请求最新建议。Gin 路由读取当前行程快照；若快照中的 Advice 尚未过期，则直接返回已有建议，否则先由数据感知 Agent 生成或更新 WorldState，再调用行动决策 Agent 进行评估。行动决策 Agent 根据当前时间、旅客所处阶段、剩余时间和关键时间约束，生成有优先级的行动建议，并计算风险等级、建议依据和必要的缓冲时间。关键数据缺失或过期时，系统将风险降级为 unknown 或 yellow，不输出无依据的确定性结论。



行动决策 Agent 只消费数据感知 Agent 生成的 WorldState，不直接访问航班、机场或地图等外部数据源。建议结果与 WorldState 一并保存到行程快照，随后由 Gin 路由返回客户端，界面展示下一步行动、风险等级、建议依据和生成时间。

4.1.3 获取行程延误风险用例实现的设计方案

旅客查看延误风险，或航班、机场、路线等状态发生变化时，用户界面向 Gin 路由发起风险查询。Gin 路由读取当前行程快照；当快照中的风险结果仍在有效期内时，直接返回风险等级、风险分数、主要原因和数据时间。当需要重新评估时，数据感知 Agent 获取并校验航班、机场、路线和旅客位置数据，识别 unknown、stale 以及冲突字段，并将处理结果形成带有数据质量信息的 WorldState。行动决策 Agent 基于最新 WorldState 评估出发时间、剩余缓冲以及航班、机场、交通和旅客状态等风险因素，生成风险等级、风险分数、风险原因和推理依据。关键数据不可用时，系统禁止返回绿色安全结论，而是返回 unknown 或 yellow，并明确提示判断的时效限制。结果保存到行程快照后返回客户端展示。

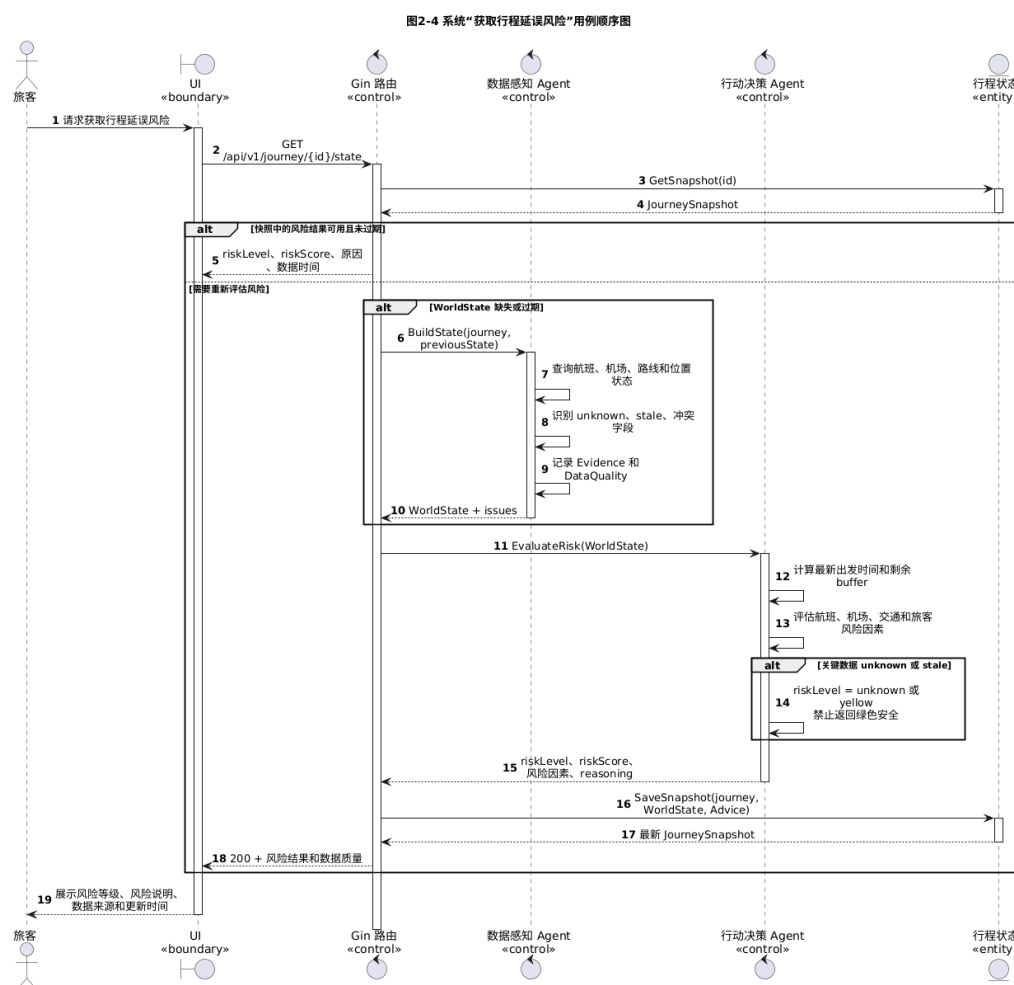
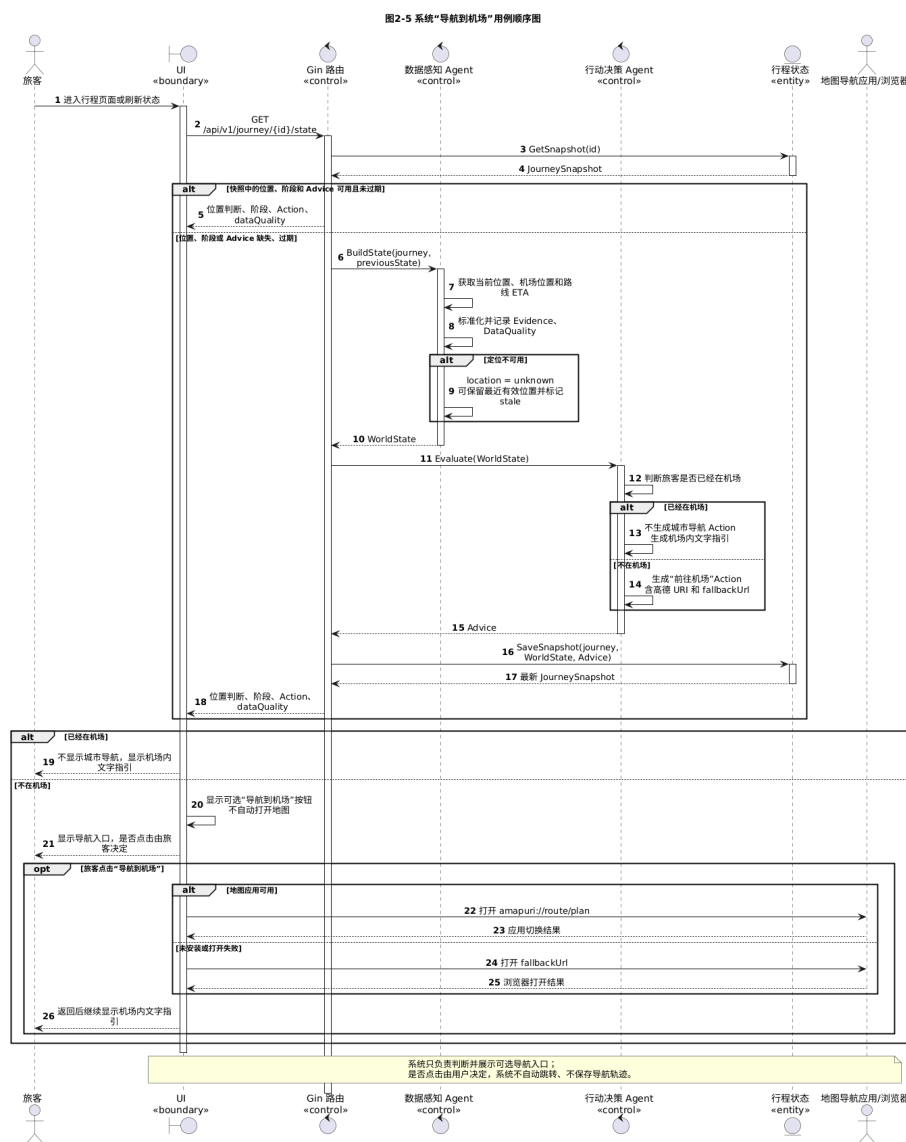


图 4-3 系统“获取行程延误风险”用例设计顺序图

行动决策 Agent 基于最新 WorldState 评估出发时间、剩余缓冲以及航班、机场、交通和旅客状态等风险因素，生成风险等级、风险分数、风险原因和推理依据。关键数据不可用时，系统禁止返回绿色安全结论，而是返回 unknown 或 yellow，并明确提示判断的时效限制。结果保存到行程快照后返回客户端展示。

4.1.4 导航到机场用例实现的设计方案

旅客在查看行程信息或行动建议后点击“导航到机场”，用户界面将请求发送至 Gin 路由。系统读取当前旅客位置、最近一次有效位置、机场目的地以及可用的导航服务，并由数据感知 Agent 对位置和路线数据进行标准化。若当前位置不可用，系统允许使用旅客手动选择的位置或最近一次有效位置，同时提示定位精度和数据时效。



行动决策 Agent 根据旅客是否到达机场决定是否生成城市导航入口：未到达时优先打开地图应用，失败时回退到导航网页；已到达时仅展示机场内值机、安检和登机口文字指引。系统只负责导航跳转，不获取或保存导航轨迹。

4.2 类设计

根据第 2 章分析类图，系统设计类按照边界类、控制类和实体类进行组织，并通过接口类连接外部数据源和服务。边界类负责接收旅客请求和展示分析结果；控制类负责请求编排、数据获取、行动决策和实时推送；实体类负责保存行程、状态、建议和快照等业务数据。系统不让行动决策 Agent 直接访问外部数据源，而是通过数据感知 Agent 形成统一的 WorldState 后再进行决策。

图4-5 系统设计类图

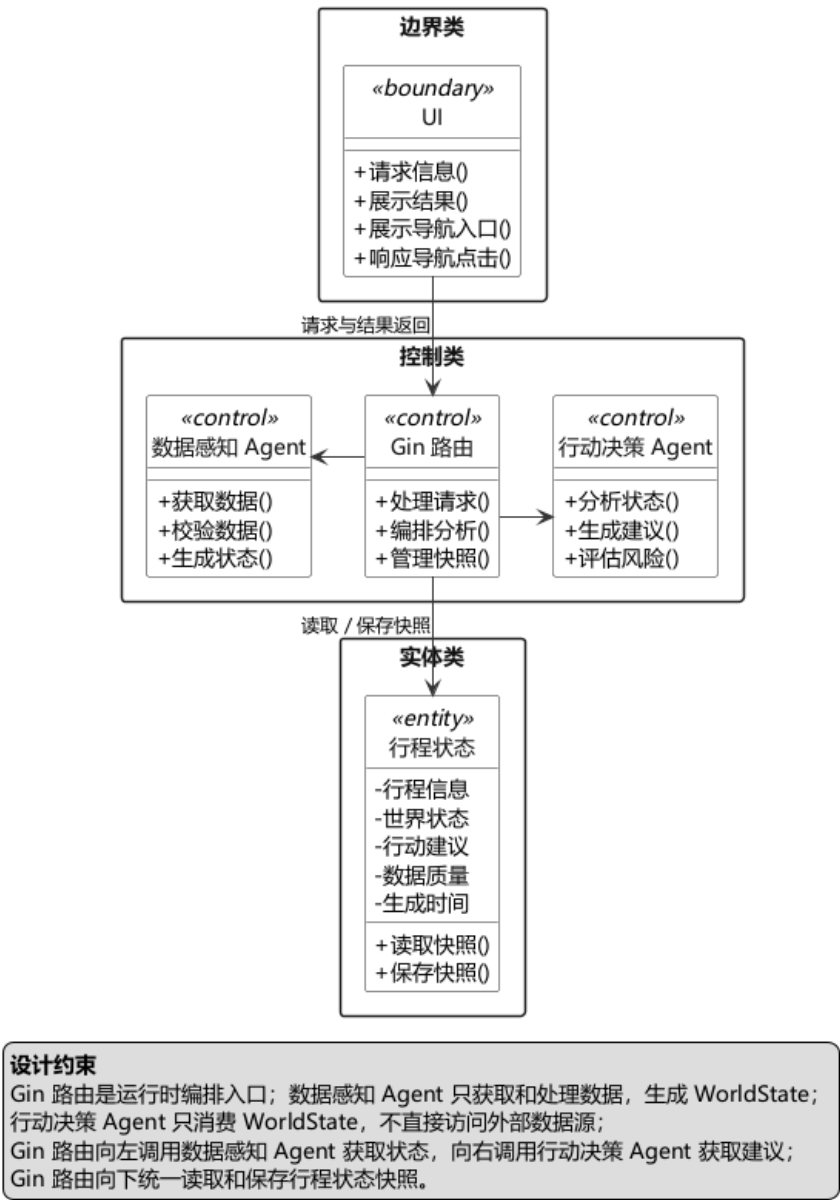


图 4-5 系统设计类图

UI 负责请求信息、展示结果和响应导航点击；Gin 路由统一编排请求并读

写行程快照；数据感知 Agent 生成 WorldState，行动决策 Agent 消费 WorldState 并生成 Advice；行程状态实体保存状态、建议、数据质量和生成时间。

表 4-1 核心设计类及职责

类别	核心类	主要职责
边界类	UI	接收旅客请求，展示行程状态、时间轴、行动卡片和导航入口。
控制类	Gin 路由、数据感知 Agent、行动决策 Agent、实时推送	统一编排请求，获取和校验数据，生成建议，并向客户端广播最新快照。
实体类	行程状态、行程快照	保存行程信息、WorldState、Advice、数据质量和生成时间。

4.3 数据模型设计

4.3.1 数据设计类及主要字段

数据模型围绕“行程快照”组织。行程实体保存旅客提交的行程标识和行李信息；航段实体保存航班号、出行日期、出发机场和到达机场；行程状态实体保存航班状态、时间轴节点、预计路程时长和数据质量；位置进度实体保存经纬度以及旅客手动确认的阶段；行程快照实体保存当前分析状态、生成时间和数据质量，并作为用户界面获取完整结果的统一对象。

表 4-2 核心数据对象及字段

数据对象	主要字段	设计说明
行程	journeyId、checkedBaggage	保存旅客创建行程时提交的基本信息。
航段	flightNumber、travelDate、departureAirport、arrivalAirport	保存单个航段的航班和机场信息，一个行程可包含一个或多个航段。
行程状态	flightStatus、timelineNodes、etaMin、dataQuality	保存航班状态、时间轴、预计路程时长和数据质量。
位置进度	longitude、latitude、manualStage	保存当前位置及旅客手动确认的出行阶段。
行程快照	analysisStatus、updatedAt、dataQuality、worldState、advice	保存一次完整分析结果，供接口返回和实时推送使用。

4.3.2 数据操作设计

系统采用进程内快照存储，不依赖数据库、Redis 或独立持久化服务。数据操作由 Gin 路由统一协调，数据感知 Agent 和行动决策 Agent 通过编排入口获取所需的行程和快照数据。主要操作如下：

- GetSnapshot(id)：读取指定行程的最新完整快照，判断快照是否存在、过期或降级。
- SaveSnapshot(journey, worldState, advice)：保存最新的行程状态、行动建议、数据质量和生成时间。
- BuildState(journey, previousState)：基于行程和最近一次有效状态获取外部数据，生成新的 WorldState。
- Evaluate(WorldState)：依据当前时间、旅客阶段、剩余时间和关键约束生成 Advice、风险等级及推理依据。
- UpdateLocationOrStage(id, location, stage)：更新旅客位置或手动确认阶段，并立即触发重新分析。
- BroadcastSnapshot(snapshot)：通过实时推送将最新完整快照广播给订阅该行程的客户端。

外部数据源单点失败时，系统保留其他数据的正常返回，并在 dataQuality、issues 和建议原因中记录降级信息。数组类型数据在无内容时返回空数组，缺失或无法确认的数据返回空值，不使用无依据的默认数字替代。

4.4 部署设计

系统采用前后端分离的轻量部署方式。前端为 Vue3 与 TypeScript 单页应用，运行在旅客的浏览器中，负责行程输入、状态展示、时间轴、行动卡片和导航入口。后端为单个 Go 可执行程序，同时承载 HTTP API、Gin 路由、数据感知 Agent、行动决策 Agent、实时推送和进程内快照存储。前后端通过 HTTPS 接口和 SSE 实时通道交互。

后端通过设备接口层访问聚合航班与机场数据源，并按需调用大模型服务；导航功能不在后端执行导航过程，而是由前端根据后端返回的导航参数跳转到地图应用或导航网页。系统不部署数据库、Redis、消息队列或独立模型服务，课程演示版的快照数据保存在后端进程内，进程重启后数据丢失。

表 4-3 软件与外界环境的交互关系

系统组成	运行环境	交互对象	交互方式
Vue3 + TypeScript 前端	旅客浏览器	Go 后端服务	HTTPS API； 通过 SSE 接收实时快照，SSE 不可用时可回退为定时刷新。
Go 后端服务	服务器或课程 演示机	航班与机场数据来源	普通 HTTPS 请求；获取航班状态、机场概况和延误相关数据。
Go 后端服务	服务器或课程 演示机	大模型服务	通过大模型接口提交 WorldState 并获取行动建议；未配置时系统仍可运行。
前端导航入口	旅客浏览器	地图导航应用或 网页	优先打开地图应用，失败时回退到导航网页；系统不采集导航轨迹。
进程内快照存储	Go 后端进程内存	业务逻辑层	保存和读取当前行程快照；进程重启后数据清空。